第 10 章 多模态与本地模型

> 本章目标：按"输入→输出"认识各类模型能做什么、会在哪失败；理解生图的主要技术路线与扩散模型为何成为主流；掌握模型选型维度、本地部署概念与显存估算；对许可证、来源与评测保持警惕。

前几章把大模型当"会写字的工具"。但 AI 还能听、能看、能画。本章把这些任务摆成一张图，再讲怎么把模型放到自己的机器上跑。

10.1 一张任务表

判断模型靠不靠谱，先问清**输入什么、输出什么**。"AI 能识别图片"——输出是"一个类别"，还是"每个像素属于什么"？

任务	输入 → 输出	一句话原理	典型用途	常见失败
LLM	文本 → 文本	逐 token 生成下一段文字	对话、写代码、Agent	自信编造、约束失守
ASR 语音识别	音频 → 文本	声学特征映射为字符序列	会议记录、字幕、口令	噪声/口音/远场下错字连篇
TTS 语音合成	文本 → 音频	先转声学表示，再还原波形	播报、有声读物、应答	多音字、数字读错、语气平板
生图	文本 → 图片	从噪声逐步去噪，逼近描述	设计草稿、素材	手指、文字、数量出错

任务	输入 → 输出	一句话原理	典型用途	常见失败
视频生成	文本/图片 → 视频	沿时间维保持一致并逐帧生成	短片、分镜预演	动作突变、物体变形
目标检测	图片 → 框 + 类别	回归物体位置框并分类	机器人视觉、安防	遮挡、小目标、类别混淆
分割	图片 → 像素 mask	对每个像素判断归属	抠图、抓取区域	边缘糊、同类粘连
深度估计	图片 → 深度图	从单图线索推断远近	3D 重建、避障	透明/反光面失效
分类	图片 → 类别	整图特征映射到标签	质检、相册归类	训练集外类别硬给答案
Embedding	文本/图片 → 向量	压缩成可算相似度的向量	搜索、知识库、聚类	相近语义排序不稳
音源分离	音频 → 多轨	按时频特征拆出声源	人声伴奏分离	音色相近串轨、残留
降噪	噪声 → 干净音频	估计噪声成分并抑制	通话增强、录音清理	人声发闷、"水下感"

> 要点：所有模型都有**适用范围**。看到演示先问：它在什么输入上成功？换到我的场景，输入差在哪？

10.2 生成图片的技术路线

"生成一张图"有好几条路，区别在于**怎么定义"像一张真实的图"**。

技术路线	核心原理	代表模型	特点
扩散 Diffusion	学习"如何逐步去噪"，从随机噪声还原图像	DDPM、Latent Diffusion、Stable Diffusion、Imagen	主流；质量高、可控性强，采样要多步
GAN	生成器与判别器对抗训练	DCGAN、StyleGAN、BigGAN	生成快；训练不稳，易模式坍塌
VAE	编码到潜空间，再从潜变量采样解码	VAE、β-VAE、VQ-VAE	潜空间直观，适合重建插值；生成偏模糊
自回归	把图像看成像素或视觉 token 序列逐个预测	PixelRNN、PixelCNN、DALL·E 1、LlamaGen、VAR	与语言模型同构，便于讲 next token
掩码生成	遮住部分视觉 token，反复预测并修正	MaskGIT	不必按顺序，能并行补全
流模型	学习从简单分布到图像分布的可逆变换	RealNVP、Glow	密度可显式计算，但结构约束强
Flow Matching	学习从噪声流向图像的速度场	Stable Diffusion 3、FLUX 相关路线	与扩散很近，是重要新路线
能量模型	学能量函数，采样寻找低能量结果	JEM 等	偏研究导向，采样与训练较复杂

为什么扩散成了主流？ 一是训练目标干净：不要求一次画对，只需判断"给定加噪程度时噪声长什么样"，比对抗训练稳定。二是**可控性强**：生成是多步去噪，每步都能被引导——文字通过条件机制影响每步，"局部重画""按骨架摆姿势""按参考图定风格"都能插进过程。潜空间扩散把计算放在压缩表示上，成本也随之下降。GAN 快但难训，自回归好但慢，扩散是折中。

ComfyUI 的节点图值得单独说。传统界面是"填提示词→出图"，隐藏中间步骤；ComfyUI 拆成节点连线：加载模型→编码提示词→采样去噪→解码→保存，每节点可换可调。它其实是**把上面这张表画成了流程图**。

10.3 语音：听与说

TTS（文本转语音）与 ASR（语音转文本）方向相反，却同样容易误判：**演示片段好听，不代表你的场景能用。**

音色克隆的边界。 技术上几十秒样本就能复制音色；伦理与法律上很清楚：**必须获得本人明确同意**，不得伪造他人声音用于冒充、诈骗或传播其未说过的观点，"内部娱乐"也不行。课堂只用自己或明确授权的样本。

情绪与语气是 TTS 质量的另一半：好的能控制语速、停顿与情绪，差的只会平铺直叙。"像不像真人"不是标准，"在你的场景里合不合适"才是。

中文 ASR 的难点，第 11 章的机械臂语音控制会正面撞上：

难点	表现
远场	一两米外，直达声与反射声混在一起
混响	回声拖尾糊住音节，短命令尤易错
噪声	风扇、音乐与人声频段重叠
中文特性	同音字、轻声儿化、口音差异大

对策不是"换个更大的模型"，而是**收紧任务**：固定口令表、限制距离、加提示音确认、失败时要求重复。把开放域识别降级成有限选项的匹配题。

10.4 模型选择：先问需求，再看参数

大 vs 小。 大模型知识广、推理强，但贵、慢、难私有部署；小模型在窄任务上常够用，且快、便宜、可控。很多"必须上大模型"的判断，其实是任务没描述清。

云端 vs 边缘。 云端免部署、随时可用，代价是网络依赖、数据出境、按量计费；边缘（本机、板卡）数据不出门、可离线、成本固定，代价是维护与性能上限。涉隐私数据或需稳定离线时，边缘往往更合理。

量化。 把权重从 16 位浮点压到更低位宽，换显存和速度。`Q4_K_M` 表示**每权重平均约 4-5 bit**，K 为分块量化，M 为中等混合精度（关键层保留更多位）。这是体积与质量的常用折中——越高越准越占内存，越低越省但输出可能变"钝"。

改行为还是补知识？ 选手段前先回答：

手段	适合什么	数据量	成本	更新频率	改的是
提示词	描述任务、格式、风格	无	最低	随时	行为（临时）
RAG	大量事实资料需引用	文档即可	低到中	高（资料常变）	知识
LoRA 微调	固定风格/格式/用语	数百到数千样本	中	低	行为（稳定）
全量微调	改变根本能力倾向	大量高质量数据	最高	极低	行为与知识

口诀：**能靠"说清楚"解决的别检索，能靠检索解决的别微调。** 微调不擅长记具体事实，让它背常变的价格表多半白费力气。

10.5 本地部署实践

三个常被混淆的工具，定位不同：

工具	定位
llama.cpp	推理引擎本体，C/C++ 实现，跨平台、可在 CPU 上跑，常作后端
Ollama	命令行服务，封装下载、运行、API，一条命令跑起来
LM Studio	图形界面，浏览模型、调参数、对比输出，学习友好

GGUF 是文件格式：把权重、量化信息、词表打包成单文件，专为 llama.cpp 这类本地推理设计。`.gguf` 多为本地跑，`.safetensors` 更通用，`.pth` 是 PyTorch 检查点。

显存/内存估算，别背机型结论，记公式：

```
模型权重占用 ≈ 参数量 × 量化位数 / 8
运行总占用   ≈ 权重 + KV cache（随上下文长度线性增长）+ 框架开销
```

符号化例子：参数 `P`（单位十亿），Q4（约 4.5 bit/权重），权重约 `P × 4.5 / 8` GB——10B 约 5.6 GB，14B 约 7.9 GB。再留缓存与框架开销：窗口翻倍，KV cache 也大致翻倍。"装得下"和"跑得动长对话"是两件事。

CPU vs GPU 的直觉：GPU 把矩阵运算并行铺开，生成快得多；CPU 也能跑，但慢到让人重估"要不要本地部署"。**放不进显存就退到内存甚至硬盘**，性能断崖下跌——最常见的挫败点。

> 要点：本地部署的第一道门槛不是"会不会装"，而是**算清楚要什么**：多少参数、多少位宽、多长上下文、能否接受每秒几个字。

10.6 去哪找模型

平台	角色
Hugging Face	模型与数据集集散地，文档最全，多数模型的原发布地
GitHub	代码与推理框架的家，模型常挂在 release 附件
ModelScope 魔搭	国内镜像，访问更稳，常有中文适配

看 model card 比看排行榜重要。重点四项：能力与训练数据来源、适用与不适用场景、许可证、量化版本的行为差异。**许可证**分两类：Apache/MIT 这类宽松协议，与常有商用限制、署名要求或用户数门槛的自定义协议，直接读仓库里的 license。

不要下载来路不明的 `.gguf` **或** `.pth`。权重文件大、格式不透明，可能被替换、夹带，或根本不是它声称的模型。只从官方仓库或可信镜像下载，核对大小与哈希。

10.7 评测：别只信榜单

Benchmark 有三个常见问题：**刷榜**（针对测试集过度优化）、**数据泄漏**（测试题混进训练数据）、**任务不匹配**（榜单考数学题，你要的是中文指令遵循）。它能筛掉明显不行的，选不出适合你的。

更可靠的是**做自己的小评测**：从真实场景挑 10 条任务，先写下"什么算合格"，再让候选模型各跑一遍、人工打分。过程朴素，但比榜单更贴近需求。记录保留：输入、模型、输出、判定理由。

课后题

1. 从 10.1 的表中挑一个非 LLM 任务，说明输入、输出和一个具体失败场景。
2. "逐步去噪"为什么比"一次生成整张图"更容易控制？结合扩散与 GAN 的差别回答。
3. 做"离线可用的中文语音指令控制"项目，判断选云端还是边缘、用 ASR 还是 TTS，并列三条可能

失败的原因。

4. 任务是"按公司最新制度回答问题，制度每季度更新"，在提示词、RAG、LoRA 中选一个并说明理由；若改成"统一输出为固定表格"，选择会变吗？
5. 找两个同类模型各读一遍 model card，比较许可证与适用场景，说明你敢不敢用在自己项目里。
6. （选做）本地模型实验：用 10.5 的公式先估算 7B、Q4 的权重占用与上下文翻倍后的增长，再准备一个 GGUF 模型，换高/低量化档位跑同一批问题，记录速度、内存与输出质量。条件不足时只做估算部分。